



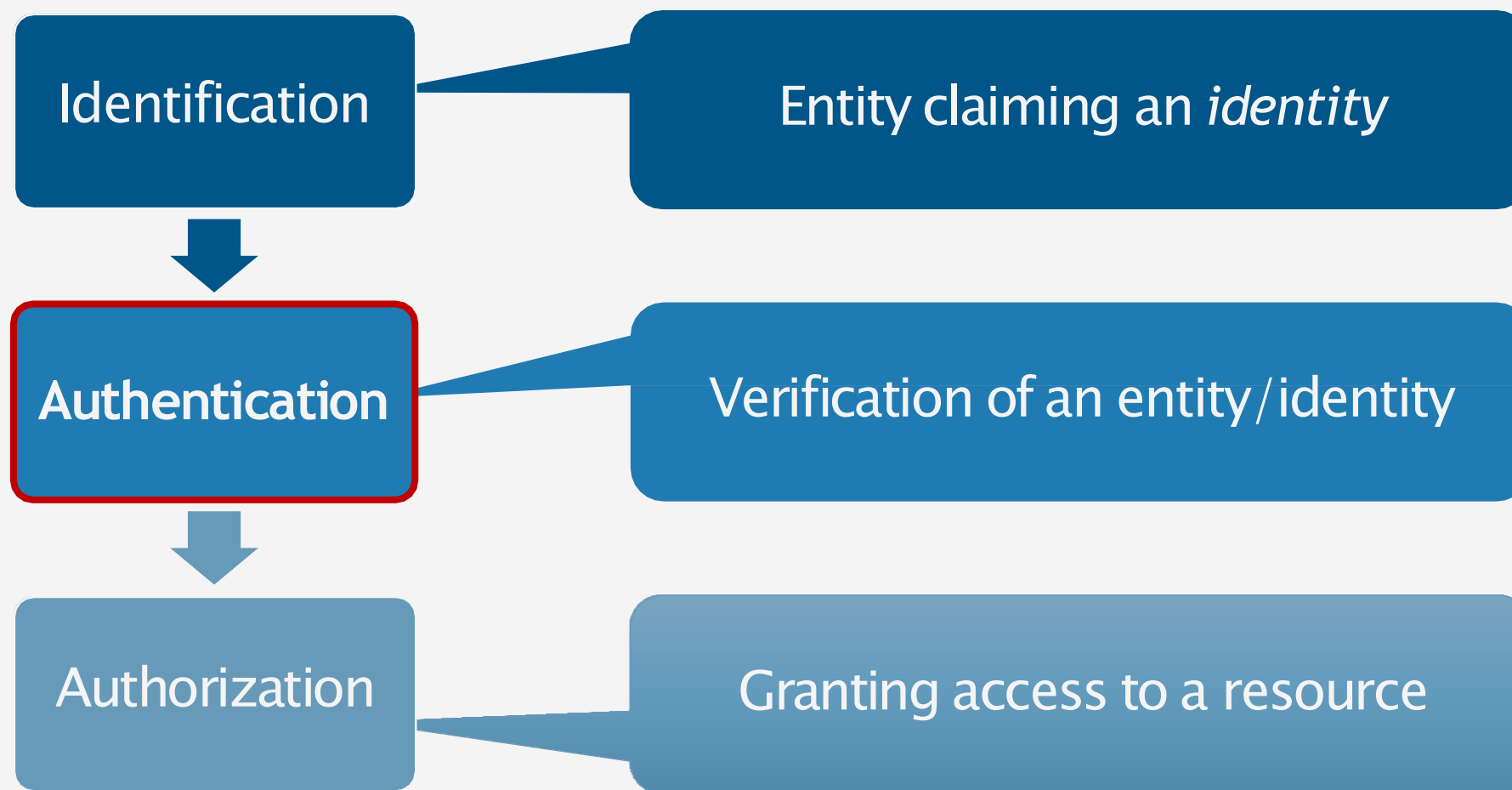
Hochschule für
Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Master Digital Transformation Cyber Security - Authentication

Prof. Dr.-Ing. Sebastian Schlesinger
Professor of Safe and Secure Cloud and
Embedded System Infrastructures



Setting the scene





Authentication

Authentication

- Confirmation of a user's authenticity
 - Verification of an active, specific entity
 - First step for authorization
- Verification of an entity





- ⦿ **Access control** is the process by which resources or services are granted or denied
- ⦿ **Identification**
 - The presentation of credentials or identification
- ⦿ **Authentication**
 - The verification of the credentials to ensure that they are genuine and not fabricated
- ⦿ **Authorization**
 - Granting permission for admittance
- ⦿ **Access** is the right to use specific resources



Authentication, Authorization, and Accounting (AAA)

- ⦿ **Authentication** in AAA provides a way of identifying a user
 - Typically with a password
- ⦿ **Authorization** determines whether the user has the authority to carry out certain tasks
 - The process of enforcing policies
- ⦿ **Accounting** measures the resources a user “consumes” during each network session



Credentials and authentication factors

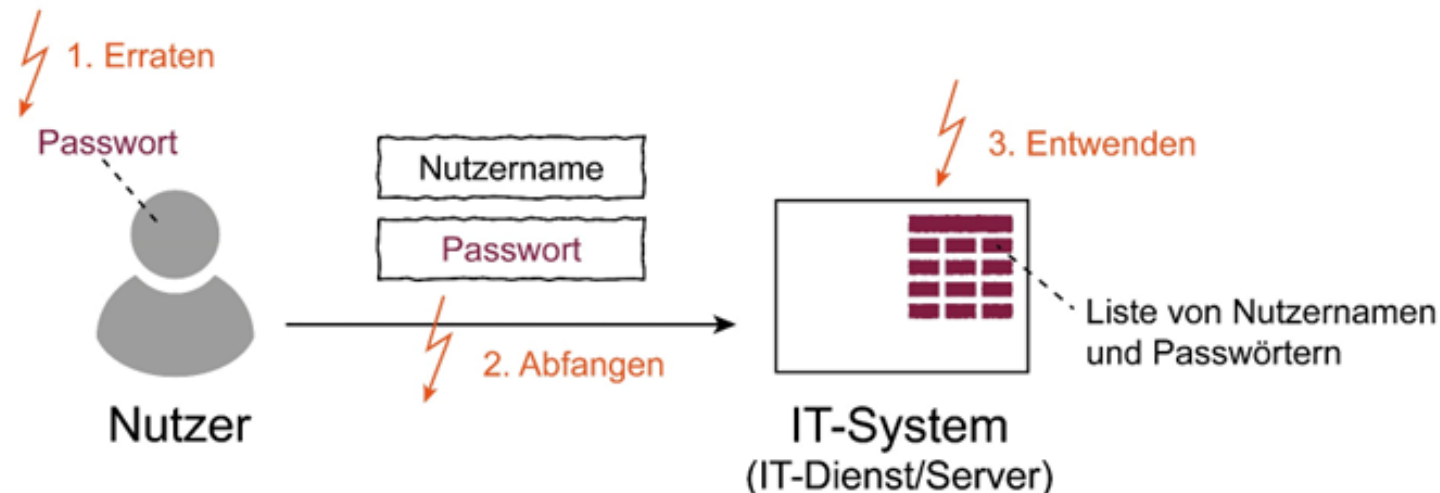
- Credentials are something you *have*, something you *are*, or something you *know*
- Types of authentication credentials
 - Passwords
 - One-time passwords
 - Standard biometrics
 - Behavioral biometrics
 - Cognitive biometrics



Some Authentication Approaches - Password



- Erraten des Passwortes durch Social Engineering oder Ausprobieren.
- Abfangen von Nutzernamen und Passwort während der Eingabe oder Übertragung.
- Entwenden der Liste von Nutzernamen und Passwörtern auf dem IT-System.



Issues with Passwords - Vector Guessing

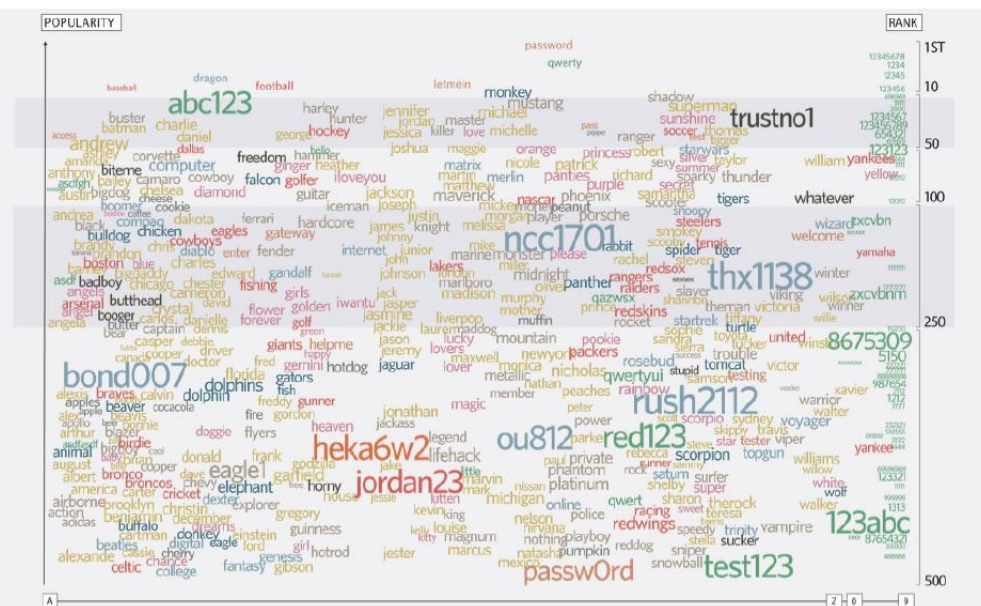


What can we do?

Die 10 beliebtesten deutschen Passwörter 2021:

1. 123456
2. passwort
3. 12345
4. hallo
5. 123456789
6. quertz
7. schatz
8. basteln
9. berlin
10. 12345678

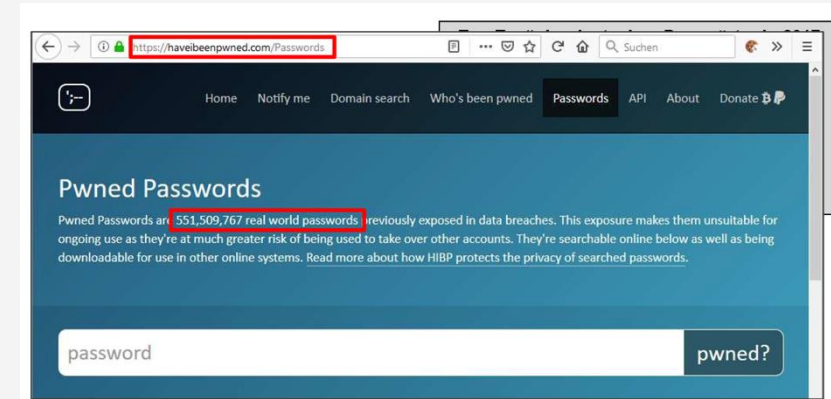
Quelle: <https://www.security-insider.de/die-beliebtesten-deutschen-passwoerter-2021-a-1084451/>



<http://www.informationisbeautiful.net/visualizations/top-500-passwords-visualized/>

Design & Concept: David McCandless
Code & additional design: Omid Kashan

Sources: data breaches, Xato.net, TroyHunt.com
Data: <http://bit.ly/KIB> Passwords



Verwendetes Alphabet	Anzahl der möglichen Zeichen	Länge des Passwortes	Anzahl der möglichen Kombinationen (vollständiger Schlüsselraum)	Zeit der vollständigen Suche
0-9	10	6	$10^6 = 1.000.000$	1 Sekunde
		8	$10^8 = 100.000.000$	100 Sekunde
		10	$10^{10} = 10.000.000.000$	2,8 Stunden
A-Z, a-z, 0-9	62	6	$62^6 = 56.800.235.584$	0,66 Tage
		8	$62^8 = 218.340.105.584.896$	6,9 Jahre
		10	$62^{10} = 839.299.365.868.340.224$	26.614 Jahre
A-Z, a-z, 0-9 ()[]? !\$%& / = * + ~ , ; : < > _	86	6	$86^6 = 404.567.235.136$	4,68 Tage
		8	$86^8 = 2.992.179.271.065.856$	94,9 Jahre
		10	$86^{10} = 22.130.157.888.803.070.976$	701.743 Jahre

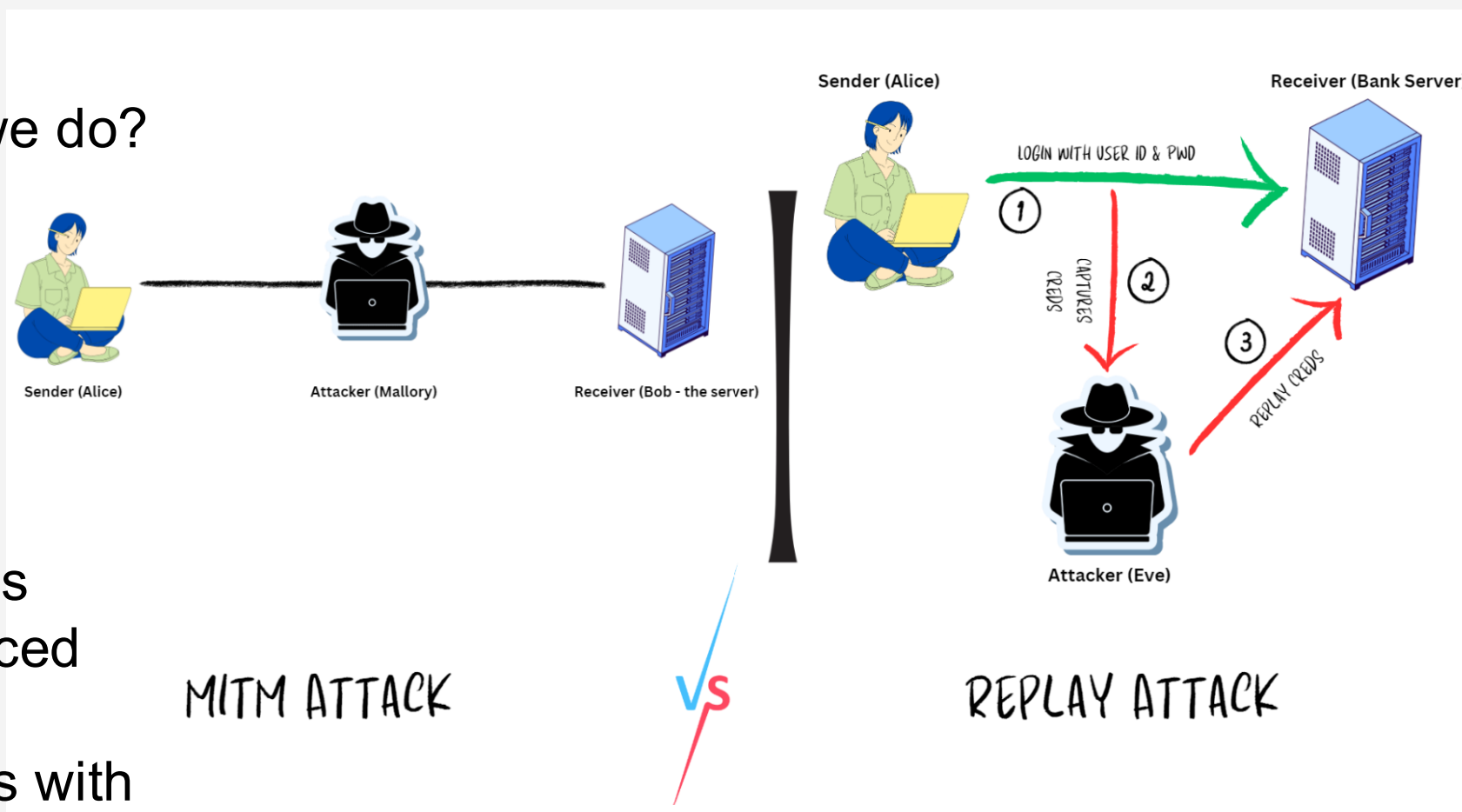
$$\text{Mögliche Kombinationen} = \text{Zeichenanzahl}^{\text{Passwortlänge}}$$



Issues with Passwords - Vector Sniffing

What can we do?

- Nonces
- MFA
- one time passwords
- Time-synced OTPs
- signatures with PKI

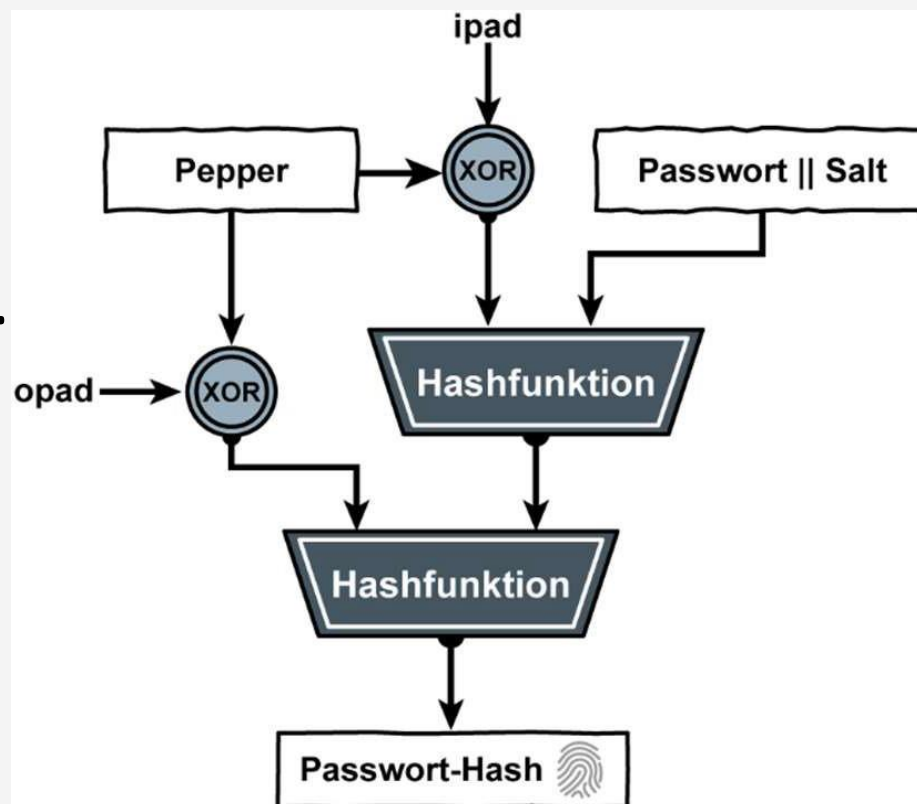


Source: <https://www.lokajittikayatray.com/post/replay-attack>



Issues with Passwords - Stealing

- password stored in clear text → salt and pepper
- keyloggers
- phishing
- spearphishing ...



Summary

Method	Examples	Security for passwords	Can be broken by
Plain text	password.txt	Completely insecure	Anyone
Hashing	md5, SHA-1, SHA-256	Very insecure	Rainbow tables Cracking Parallel cracking Predictive cracking Human error
Hashing + Salting	SHA-1 with salt SHA-256 with salt	Very insecure	Cracking Parallel cracking Predictive cracking Human error
Hashing + Salting + Key stretching	Repeated MD5	Insecure	Parallel cracking Predictive cracking Human error
	Repeated HMAC-SHA256	Nonstandard practice	
	PBKDF2-HMAC-SHA256	Still good	
Hashing + Salting + Memory-hard key stretching	bcrypt	Good practice	Predictive cracking Human error
	scrypt	Good practice	
	Argon2	Best practice	
Usable security innovations	Password managers	Best practice	Human error
	Security keys	Best practice	



Other Password Aspects

- Password Recovery
- Password Change Notification

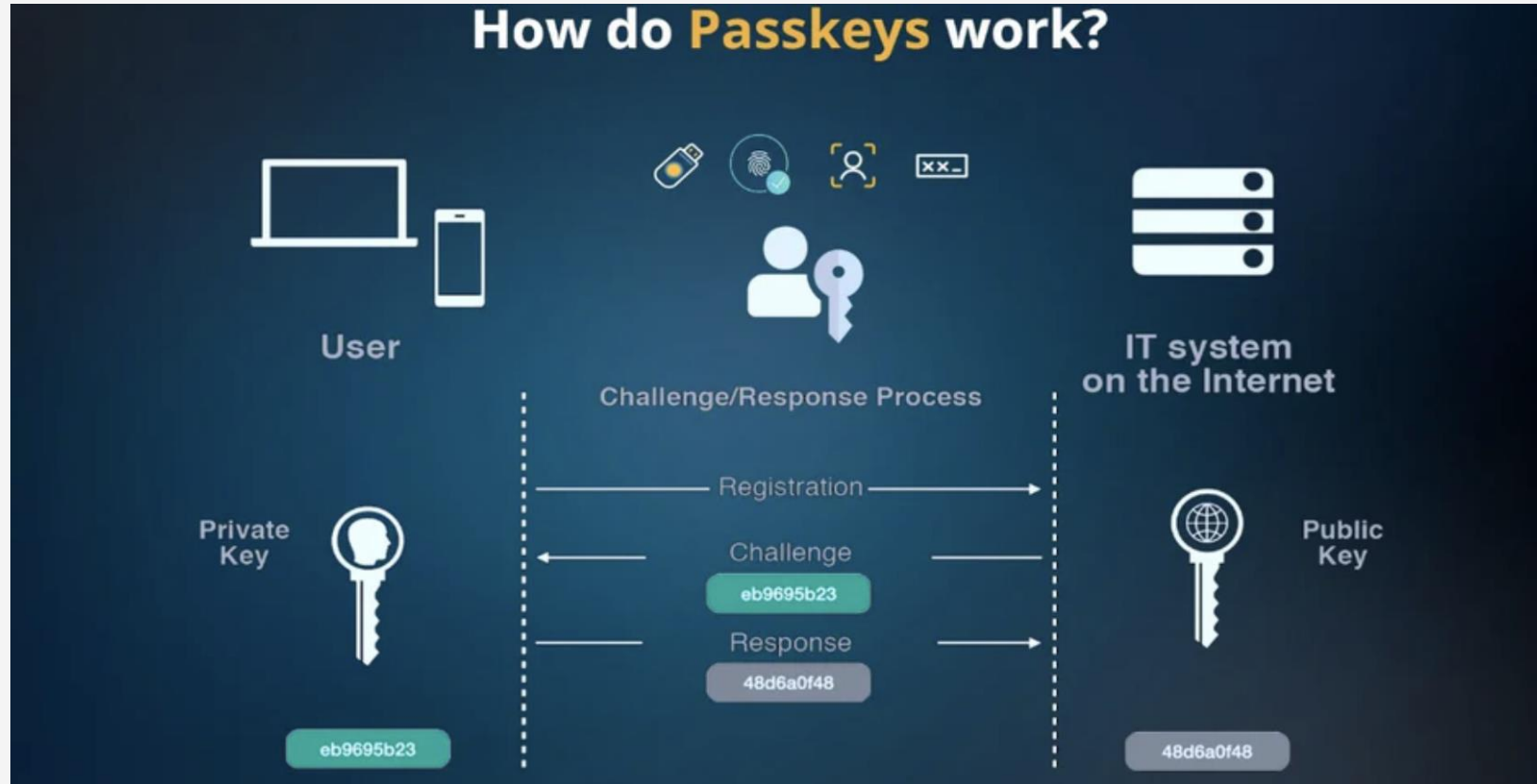
Some General Issues

- Awareness / Education
- Credential Rotation
- Password Management at Enterprises

Passkeys

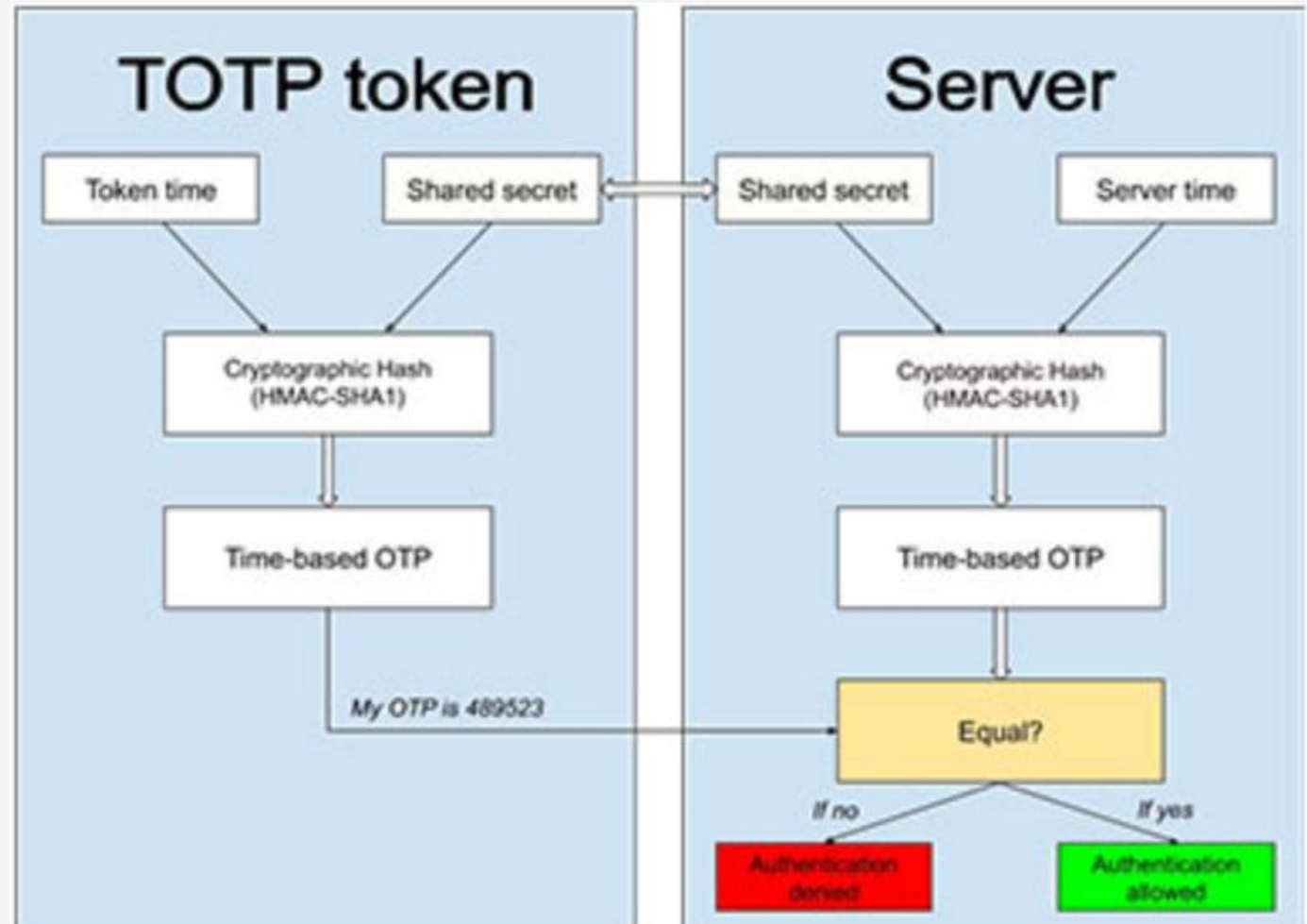


How do Passkeys work?



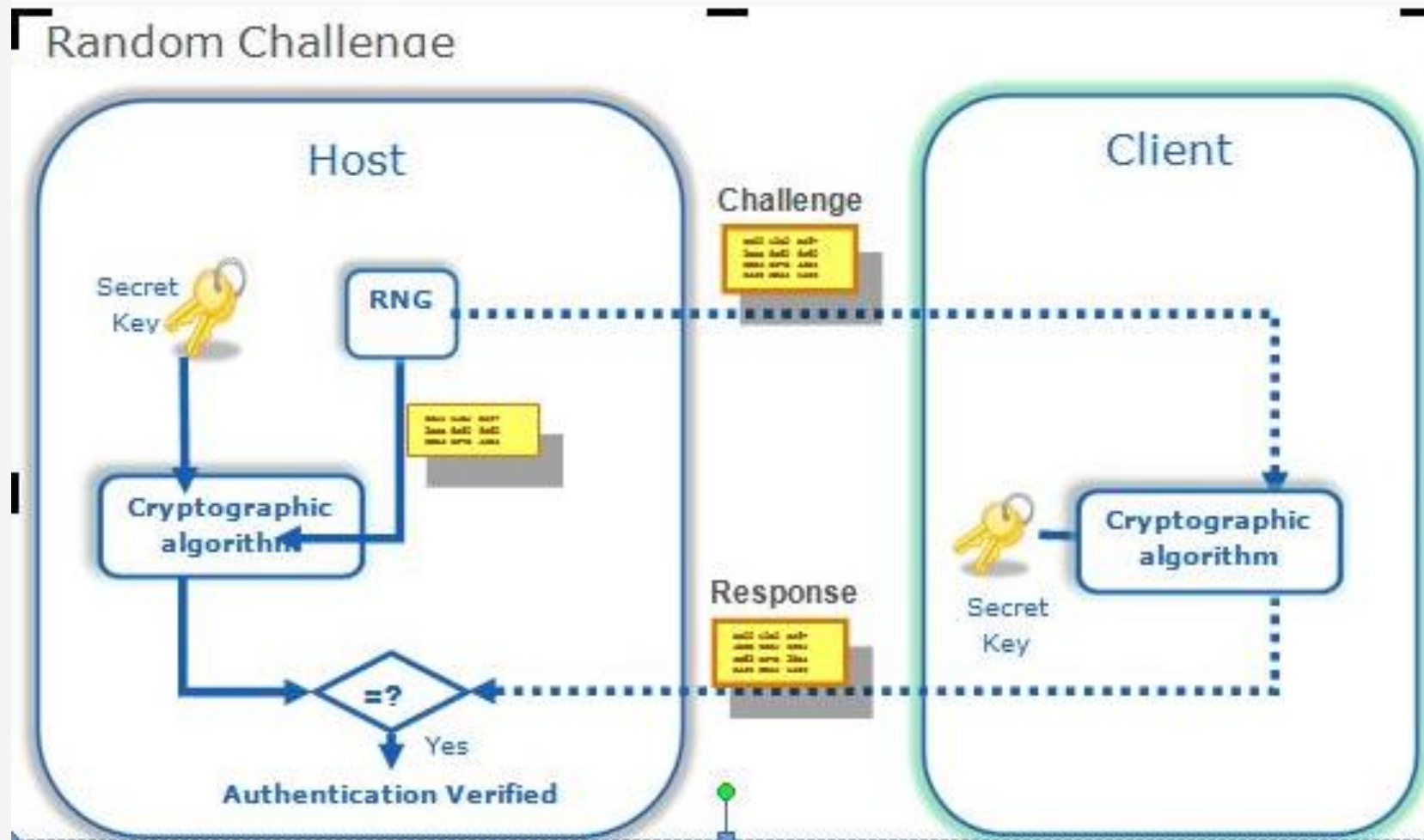
- Implicit MFA built in
- Public key cryptography built in

Time-based OTP



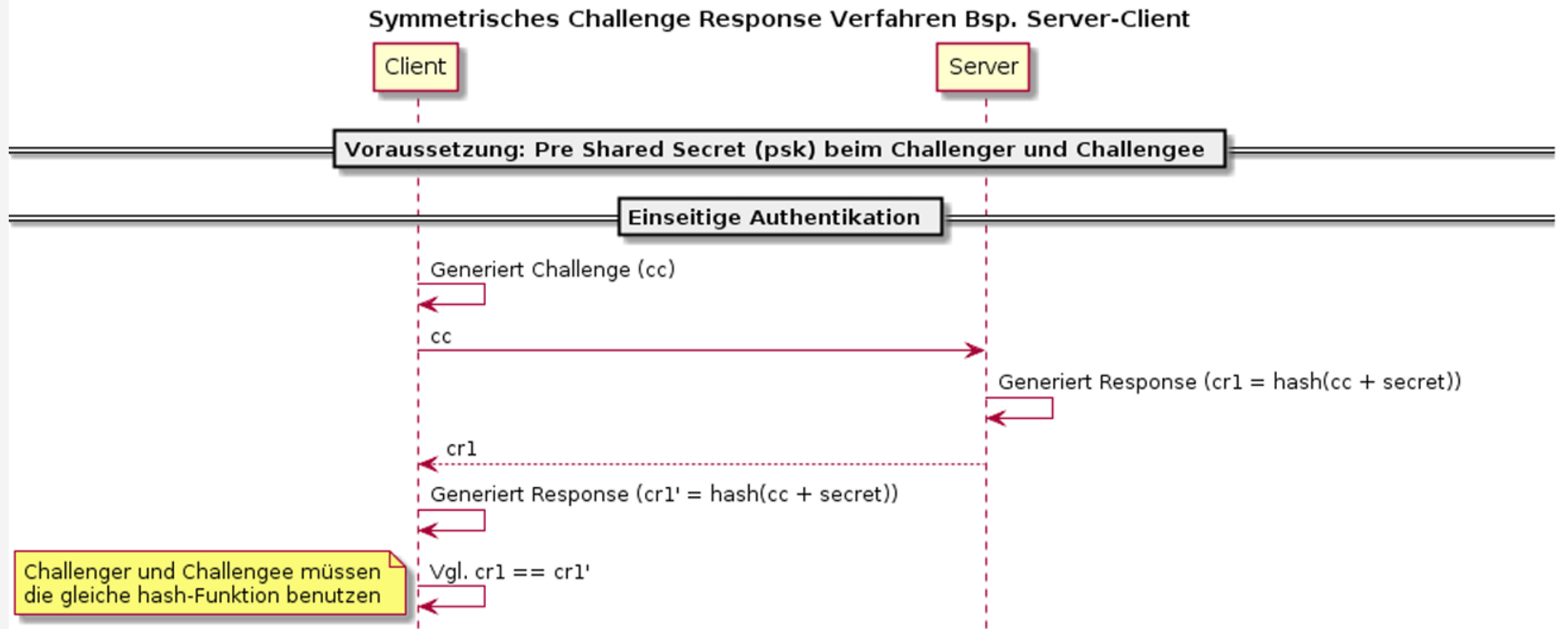


Challenge-Response Approaches



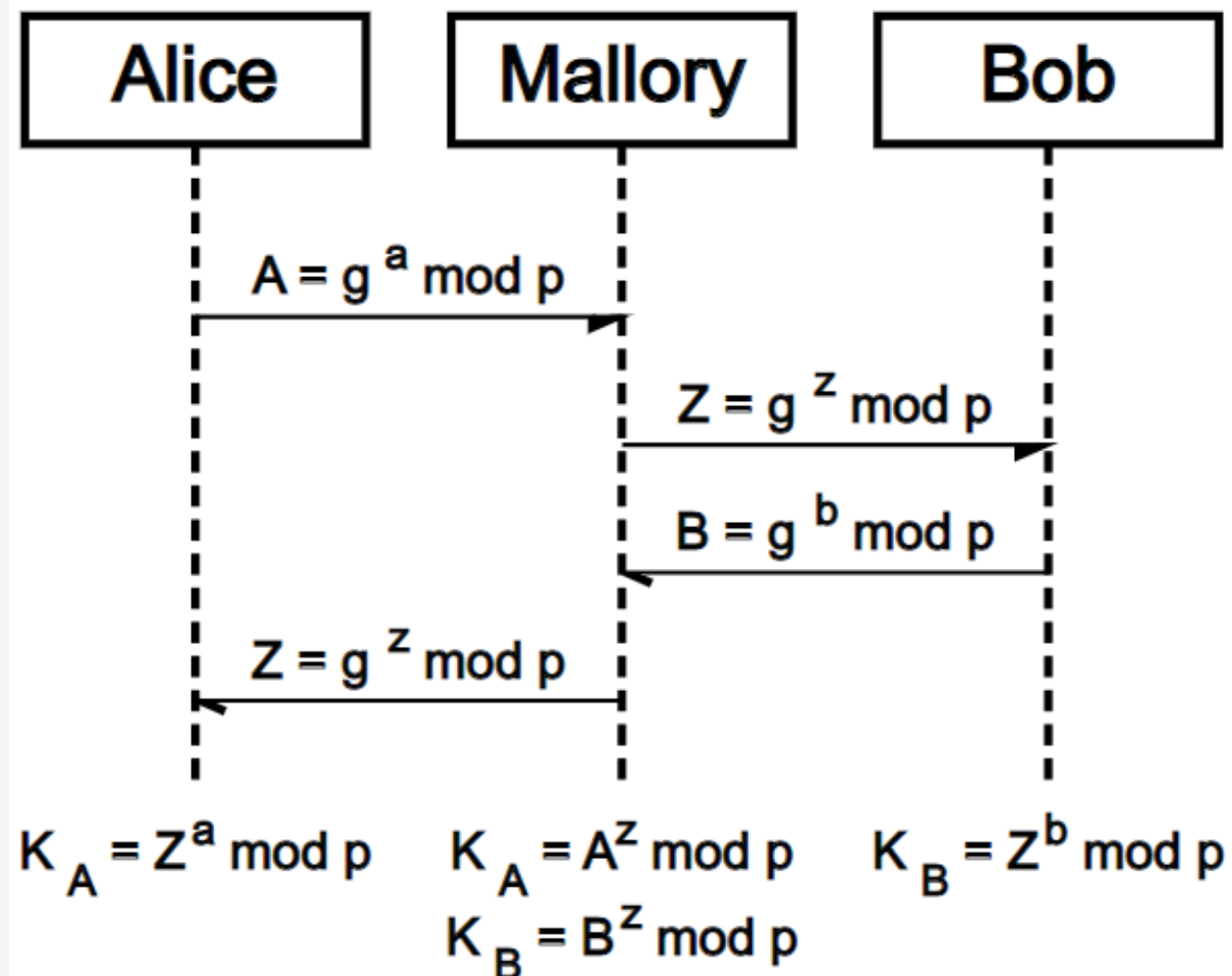
→ protection against
replay attacks

Challenge-Response with symmetric secrets



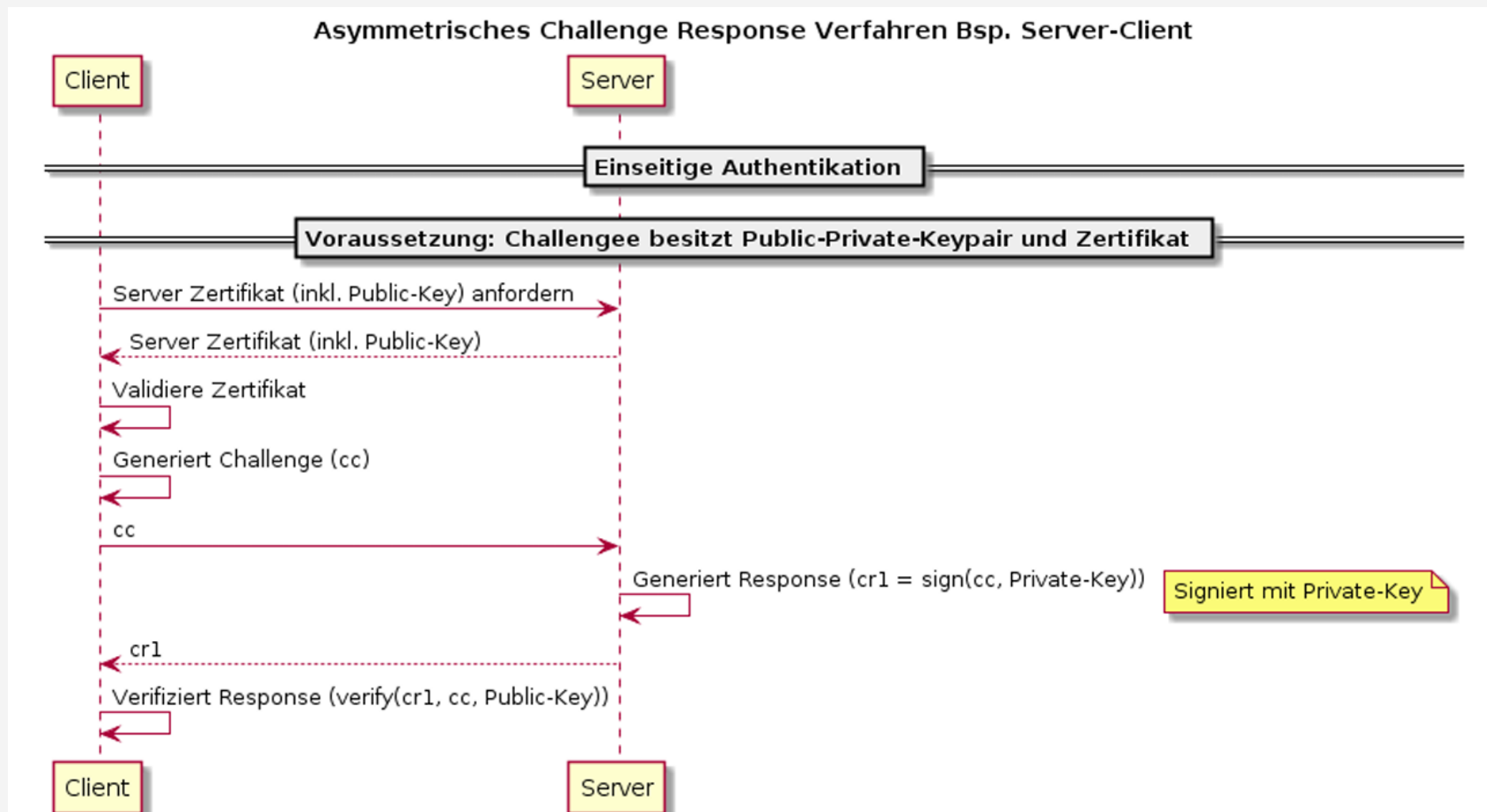


Key Exchange - Diffie Hellman





Challenge-Response with asymmetric keys

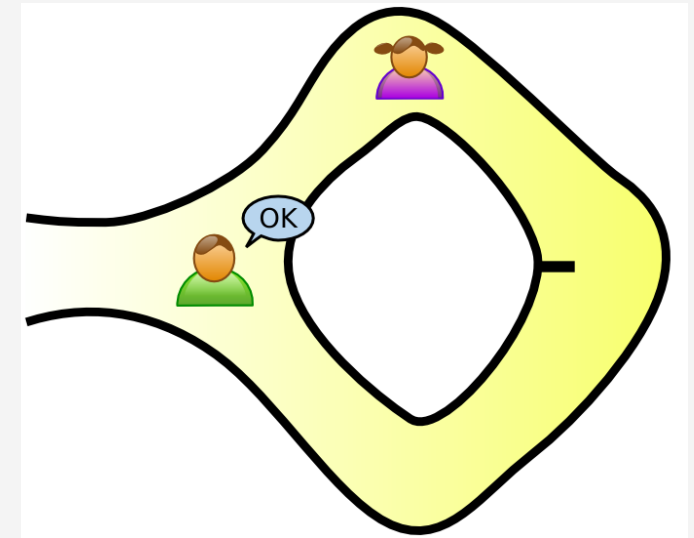
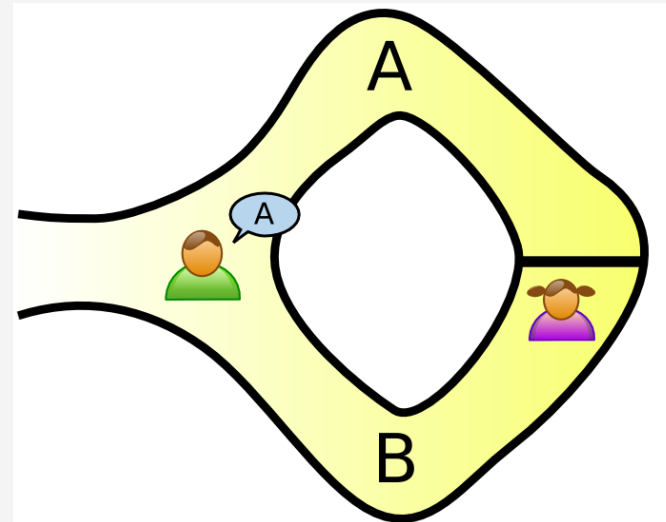
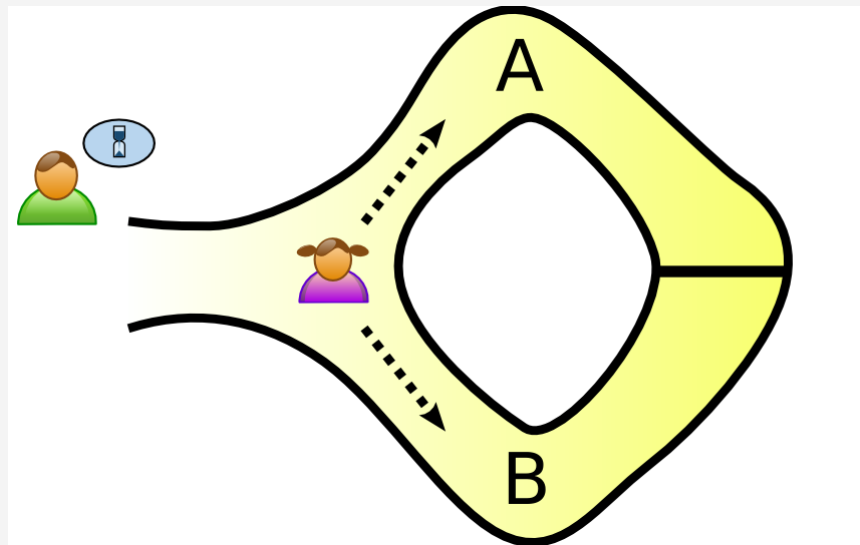


Zero Knowledge Proofs



Hochschule für
Wirtschaft und Recht Berlin
Berlin School of Economics and Law

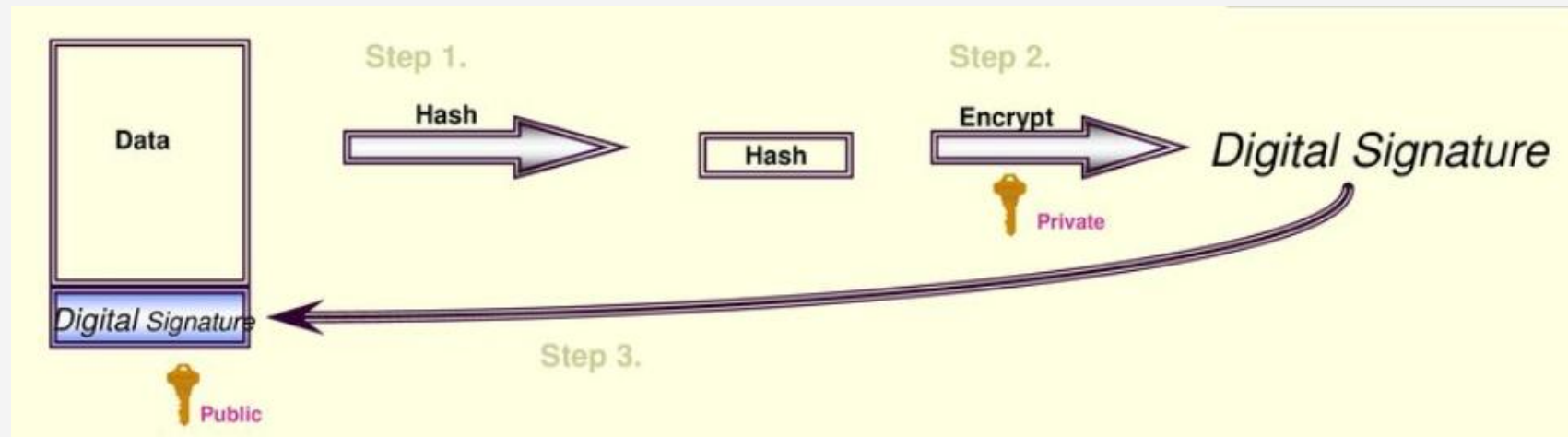
Ali Baba's cave



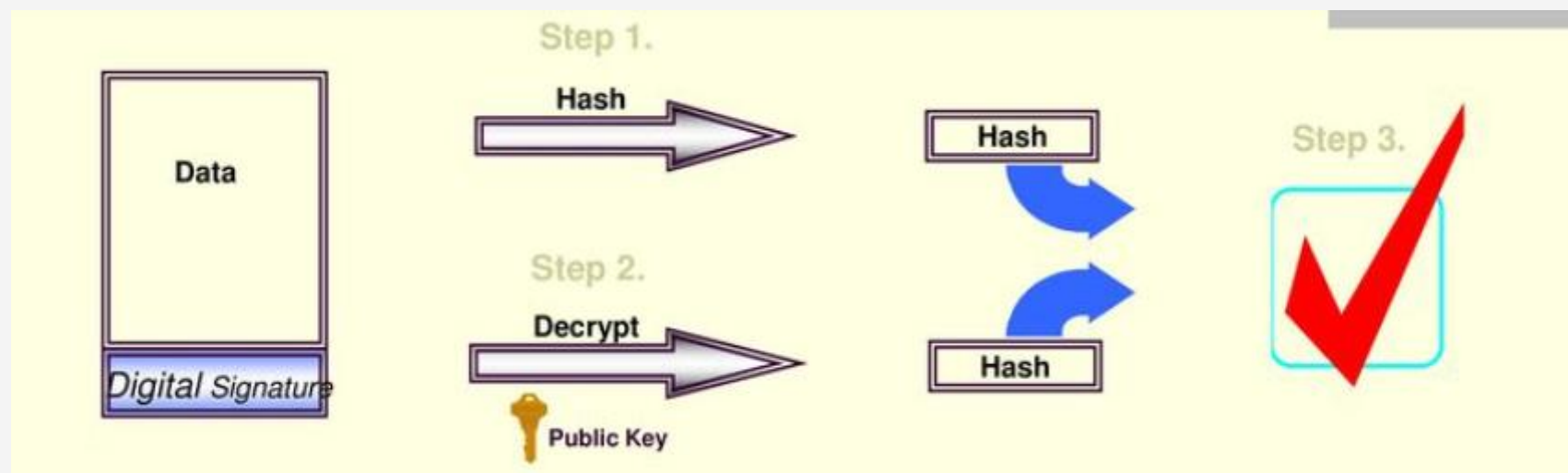


Digital Signatures

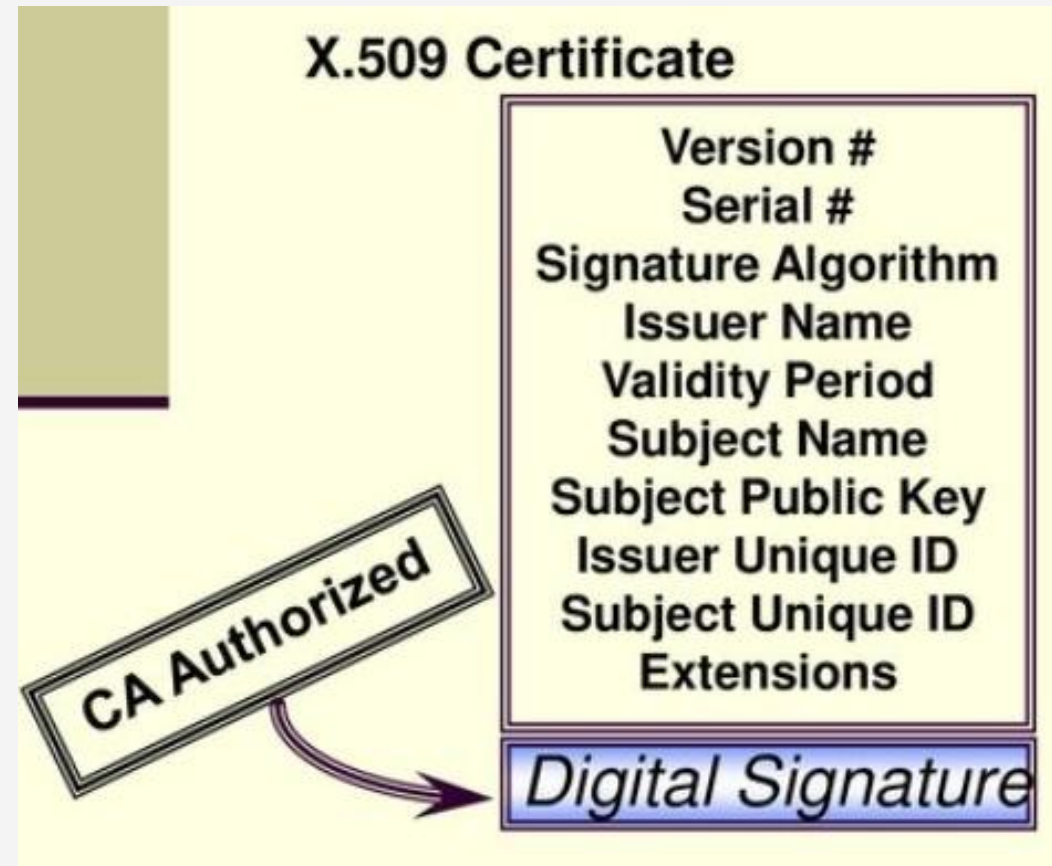
Creation



Verification

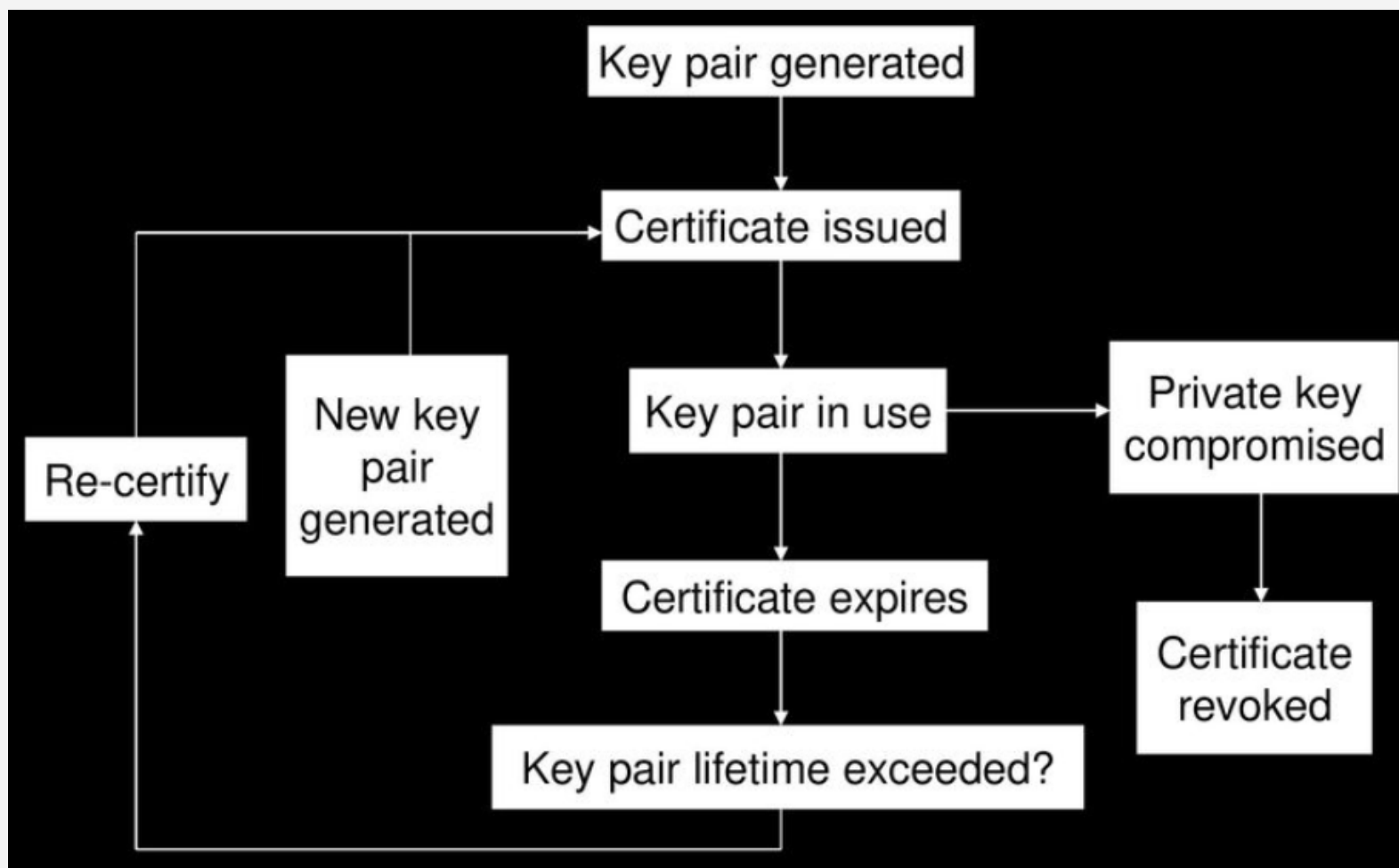


Certificates formally defined





Certificate Lifecycles





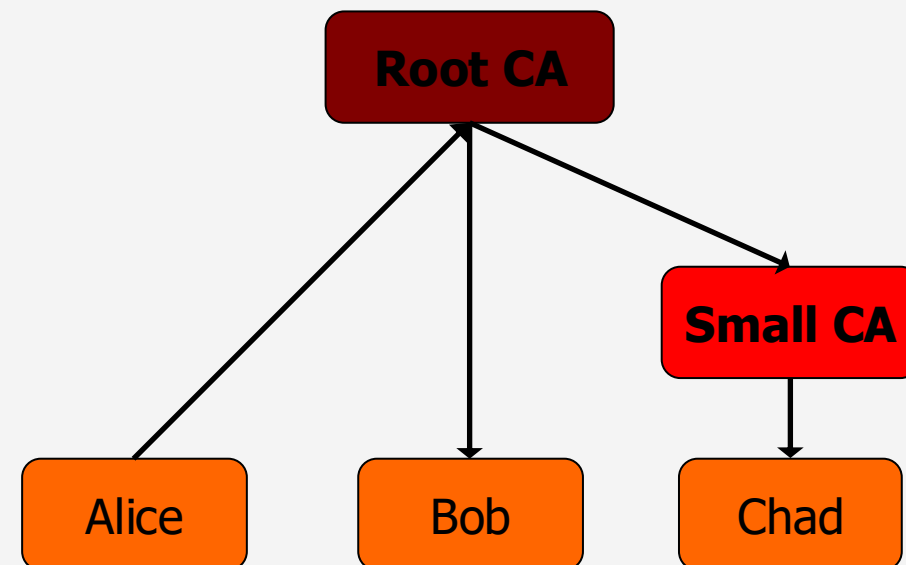
What is trust in this context?

- Trust is based on real world contractual obligations between a 3rd Party and users
- This Trusted 3rd Party is referred to as a Certificate Authority (CA)
- In other models trust is based on personal relationships that don't have a contractual basis (e.g. PGP)
- Users may allow a CA to delegate their trust
- This delegation of trust is what allows us to build large PKI's



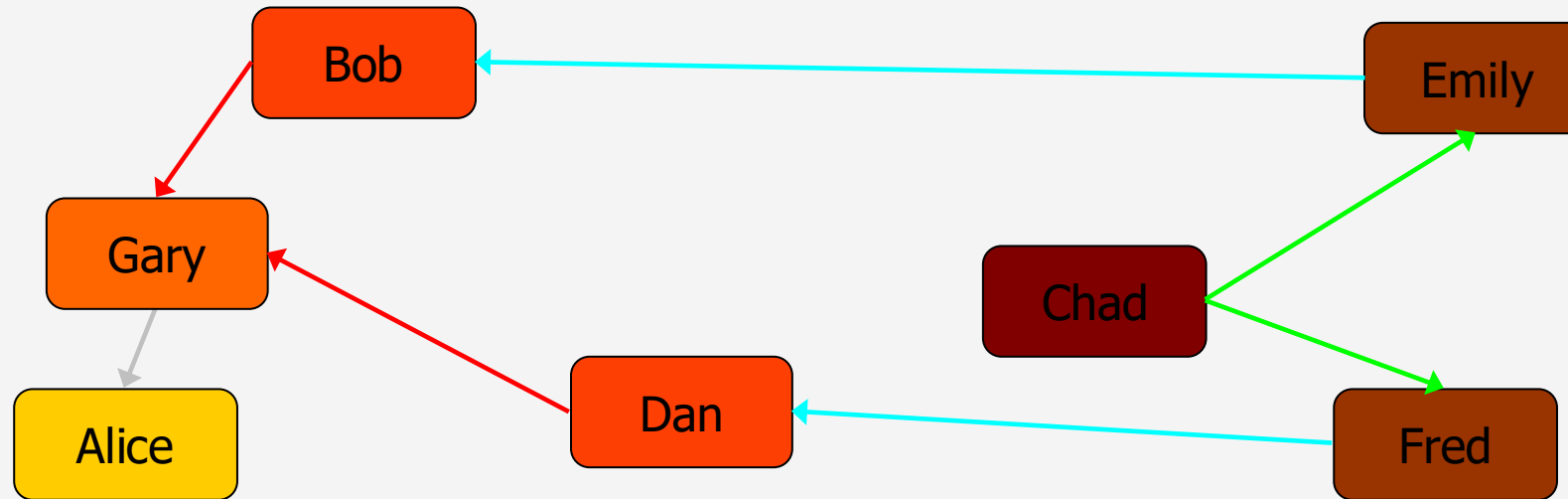
PKI

- Registration Authority (RA) to identify users
- Certification Authority (CA) to issue certificates
- Repositories to hold certificates
- Revocation Lists to keep the compromised certificates
- If Alice trusts Root CA then she trusts Bob's Certificate signed by Root CA
- If Alice trusts Root CA to delegate her trust to others then she trusts Chad's Certificate signed by Small CA





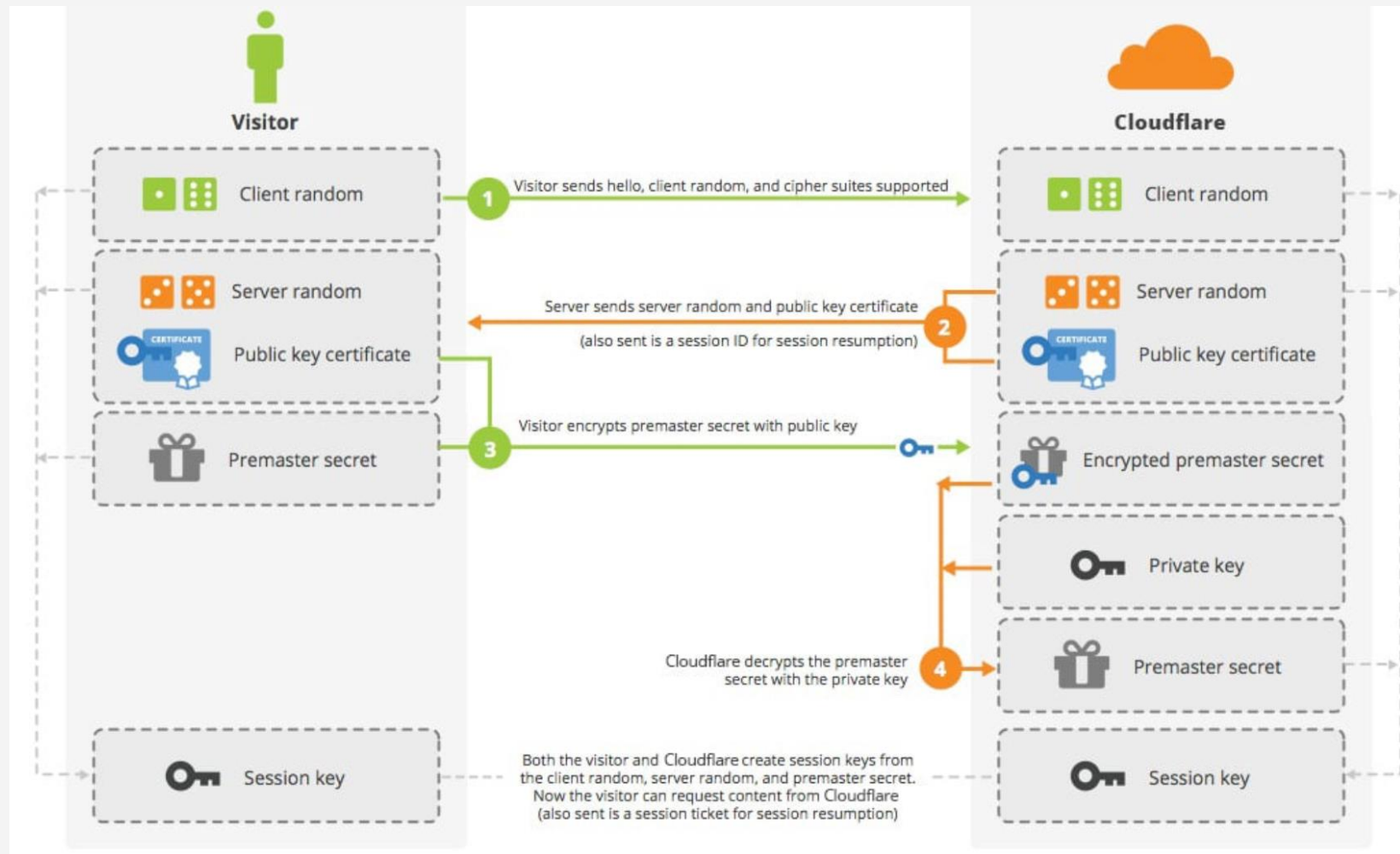
Alternative - Pretty Good Privacy (PGP)



- Each user's Certificate is signed by zero or more other users
- Certificate validity calculated from levels of trust assigned by signers
- Assigned levels (Chad)
 - Implicit: User themselves – Chad
 - Complete: Any Certificate signed by the user themselves – Fred and Emily
 - Intermediate Calculated Item
- Partial Trust: Any Certificate signed by a 'Complete' Certificate – Bob and Dan
- Calculated (Chad)
 - Valid: Any Certificate signed by an 'Implicit' or 'Complete' level Certificates – Chad, Fred, Emily, Dan, and Bob
 - Marginally Valid: Any Certificate signed by two or more 'Partial' trust Certificates – Gary
 - Invalid: Any Certificate signed by a 'Marginally Valid' or no one - Alice

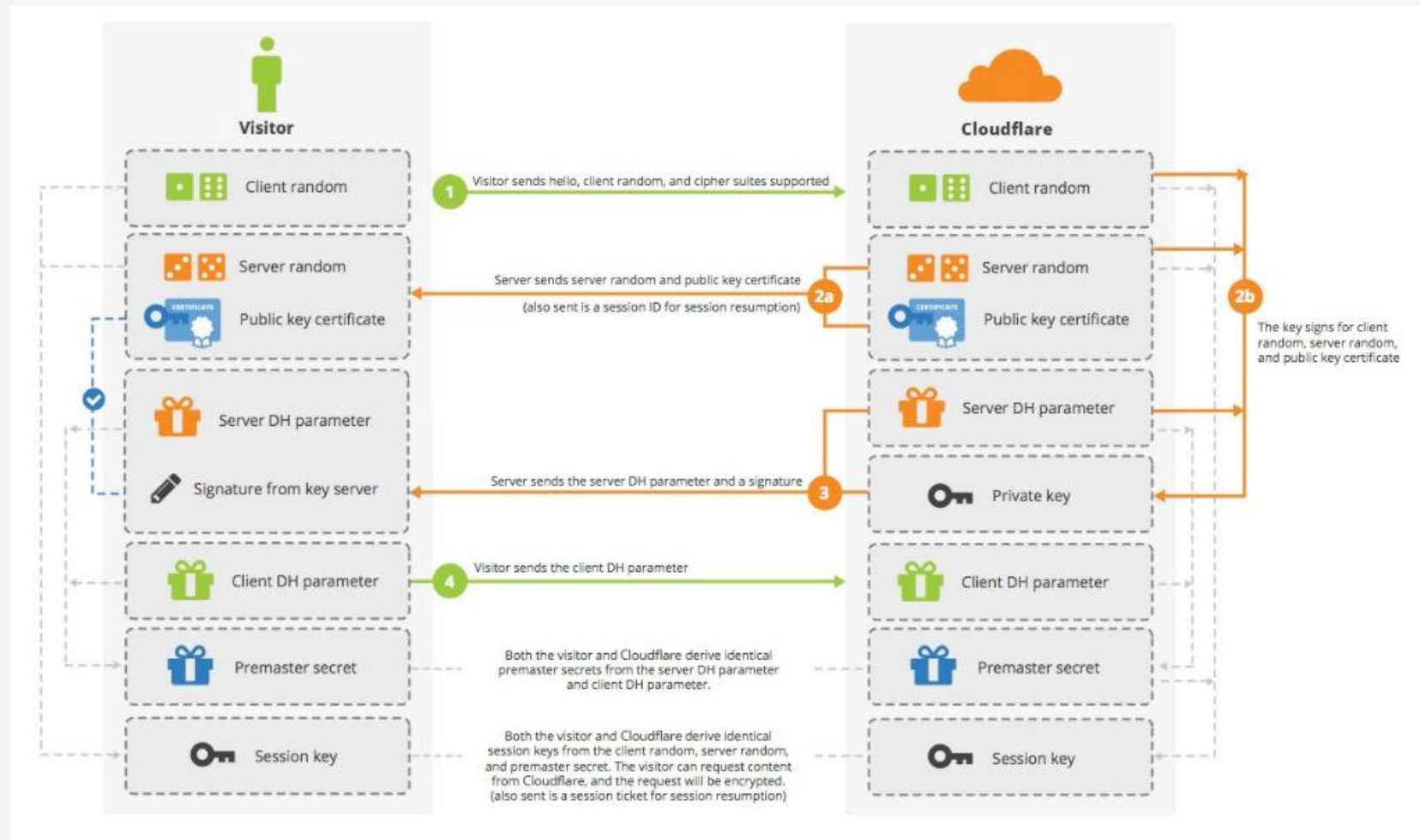


TLS Handshake in principle - RSA only





TLS Handshake in principle - with RSA and DH





Forward Secrecy

- Forward secrecy ensures that encrypted data keeps encrypted even if the private key gets compromised (leaked)
- Also known as “Perfect Forward Secrecy”
- If only RSA is used → if private key gets leaked later; attacker can decipher old messages later → no forward secrecy
- If DH is used to create an **ephemeral** session key → forward secrecy

t-tls: encrypting data



- recall: TCP provides data *byte stream* abstraction
- Q: can we encrypt data in-stream as written into TCP socket?
 - A: where would MAC go? If at end, no message integrity until all data received and connection closed!
 - solution: break stream in series of “records”
 - each client-to-server record carries a MAC, created using M_c
 - receiver can act on each record as it arrives
- t-tls record encrypted using symmetric key, K_c , passed to TCP:

K_c (length | data | MAC)

t-tls: encrypting data (more)



- possible attacks on data stream?
 - *re-ordering*: man-in middle intercepts TCP segments and reorders (manipulating sequence #s in unencrypted TCP header)
 - *replay*
- solutions:
 - use TLS sequence numbers (data, TLS-seq-# incorporated into MAC)
 - use nonce

Factor Biometry - Example Keystroke Dynamic



Hochschule für
Wirtschaft und Recht Berlin
Berlin School of Economics and Law

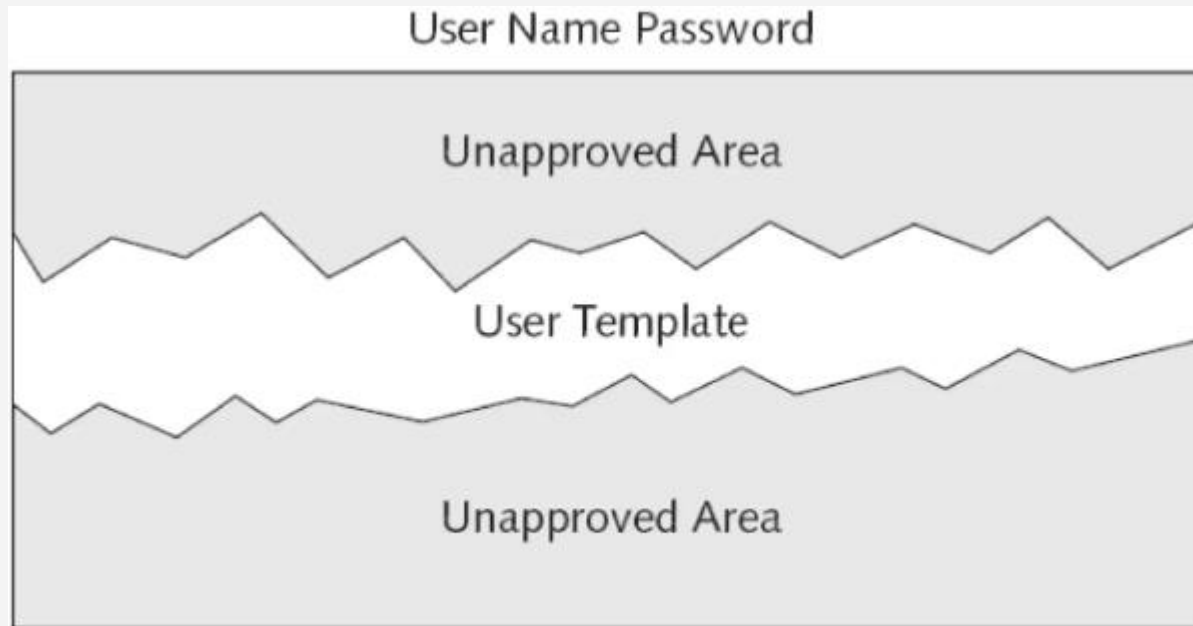


Figure 8-4 Typing template

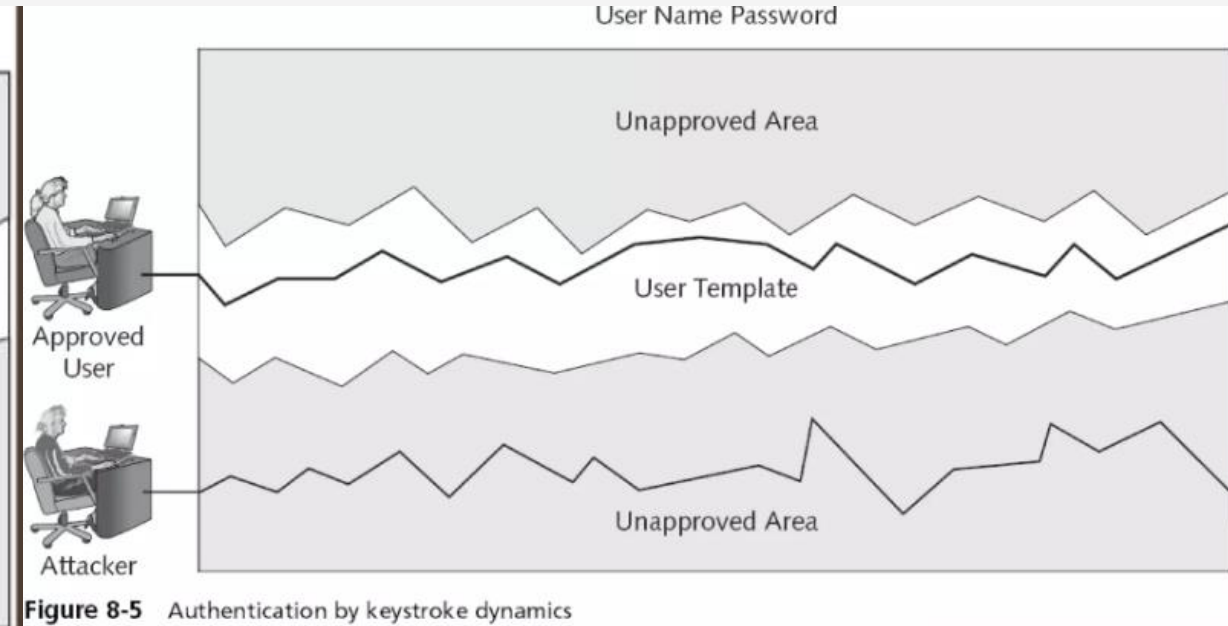
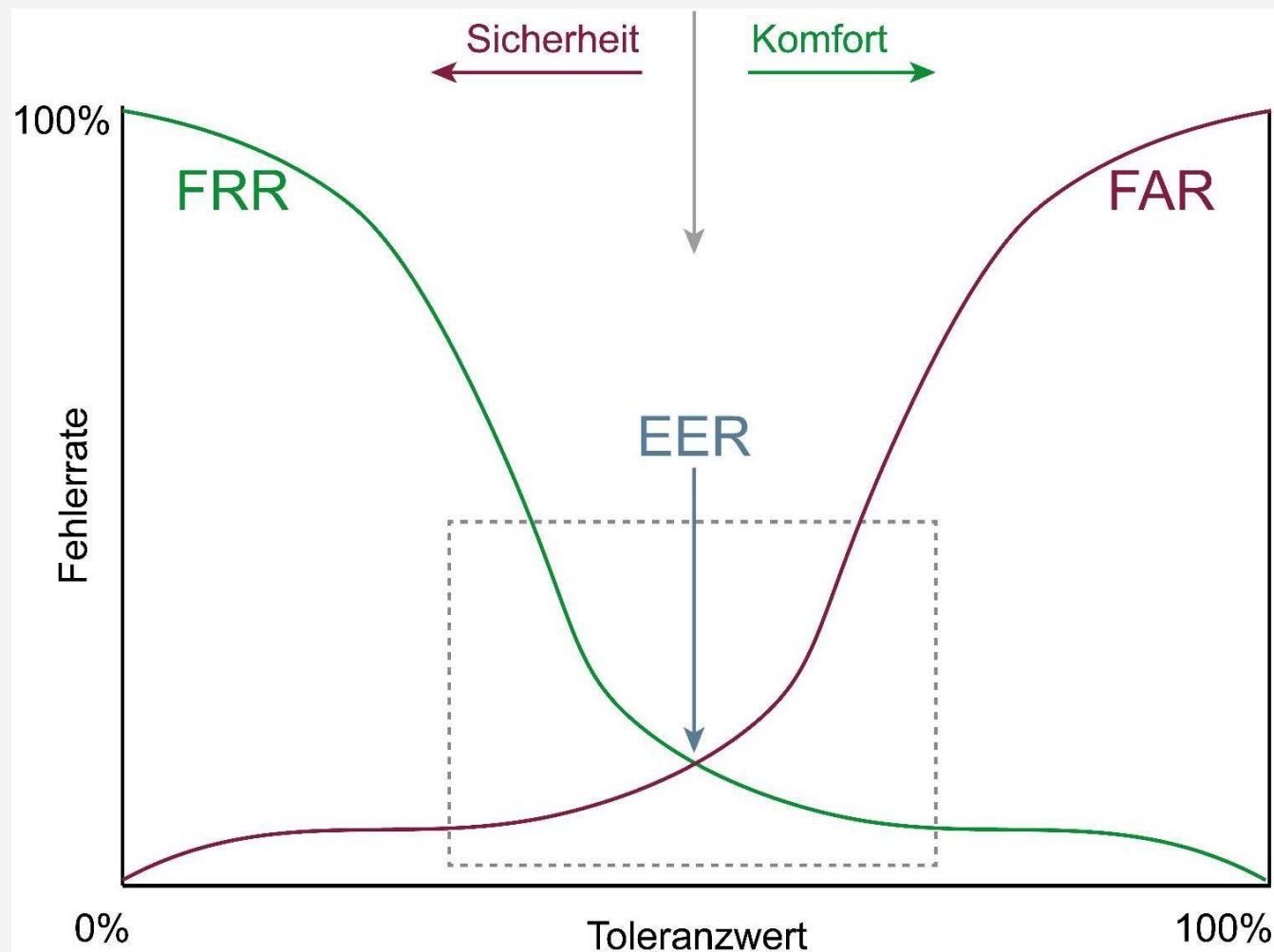


Figure 8-5 Authentication by keystroke dynamics



Factor Biometry - Challenges



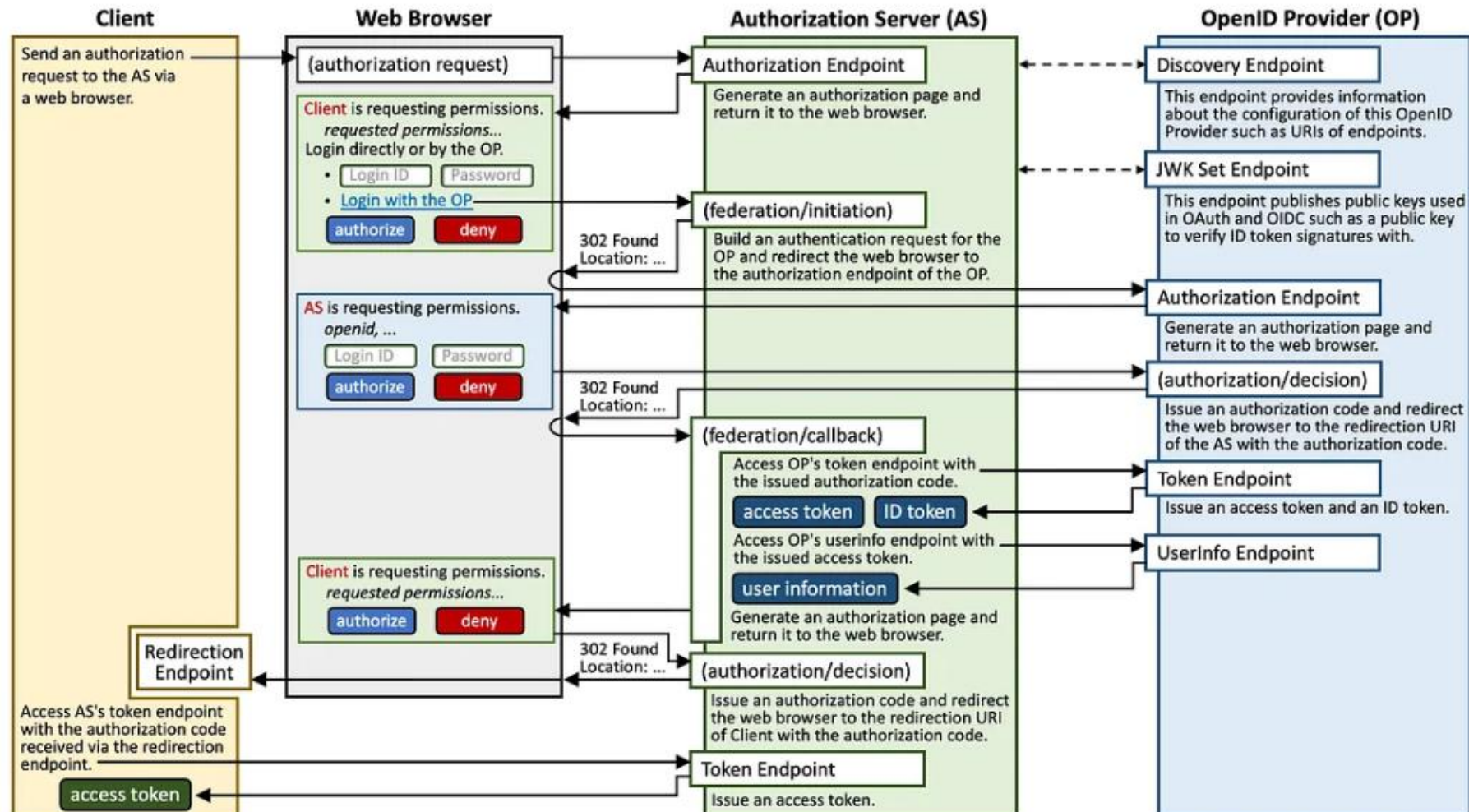


Factor Biometry - Security vs Convenience

Rank	Accuracy	Convenience	Cost	MOC integration
1	DNA	Voice	Voice	Finger
2	Iris	Face	Signature	Voice
3	Retina	Signature	Finger	
4	Finger	Finger	Face	
5	Face	Iris	Iris	
6	Signature	Retina	Retina	
7	Voice	DNA	DNA	



OpenID Connect and OAuth



```
{
  "iss"      : "https://c2id.com",
  "sub"      : "alice",
  "aud"      : "s6BhdRkqt3",
  "nonce"    : "n-0S6_WzA2Mj",
  "exp"      : 1311281970,
  "iat"      : 1311280970,
  "acr"      : "http://loa.c2id.com/high",
  "amr"      : [ "mfa", "pwd", "otp" ]
}
```

- ## Structure of a JSON Web Token (JWT)
- Header**

Payload

Signature

↓

{
 "alg": "HS256",
 "type": "JWT"
}

↓

{
 "sub": "I234567890",
 "name": "George
White",
 "admin": true,
 "iat": 1516239022
}

↓

Base64URLSafe(HM
ACSHA(<header>, <
payload>, <secret key>
>>
- eyJhbGciOiJSUzI1NiIsImtpZCI6IjFlOWdkazcifQ.eGowImlzczyI6ICJodHRwOiwvc2VydmcVyLmV4YW1wbGUyY29tIiwiaWkiCiJzdWIiOiAiAimjQ4Mjg5NnZxMDAxIiwicKICJhdWQiOiAiAiczZCaGRSa3F0MyIsCiAibm9uY2UiOiAiBi0wUzZFv3pmBmk1qTi wKiCJleHAiOiAieMzMExMjc5OTcwLAogImhhbmhlcCI6IDEzMTYODAE5bnAKfkQ.ggW8hz1EuVLuxNuuiJKX_V8a_OMxzR0EHR9R6jdgdqrOOOF4daGU96Sr_P6qpP6lcmD3HP99ObilPRs-cwh3LO-p146waJ8Ihehcwl7F09Jdi jmbqkvPeB2T9CNjqGePgpgccmg4vfKjkM8FcGvnzZUN4_KSP0aaPltoJl zzwgj xGB yKH iOt X7Tp dQyHE5l cMiKPXF EI QIL V qOp c_E2 Dz L7emopWoao ZTF_m0_N0 YzFC6 gE JboEO rOS K5hoDalrcvRYLSrQA ZZkfly uVCyx eOV9 GfNQ C3_osj zw2Paith fubEEBLuVV k4XUVrWO LR lL0 n x7 Rrk U8NX NHq- rvKMzqqg
- [Header] . [Claims] . [Digital Signature]



What is OAuth?

From the specification RFC 6749:

“The OAuth 2.0 authorization framework enables a **third-party application** to obtain **limited access** to an **HTTP service**, either on **behalf of a resource owner** by orchestrating an approval interaction between the resource owner and the HTTP service, or by allowing the third-party application to obtain access on its own behalf. “

In other words:

OAuth 2.0 is a **delegation protocol** that lets people **allow applications** to **access things on their behalf**.



- **Authorisation code flow**
 - for typical web and mobile apps
 - the client is typically authenticated
 - tokens retrieved via back channel
- **Implicit flow**
 - for JavaScript applications that run in the browser
 - the client is **not** authenticated
 - tokens returned via front-channel, revealed to browser
- **Hybrid flow**
 - allows app front-end and back-end to receive tokens independently
 - rarely used

Authorization Code Flow (RFC 6749, 4.1)

